Lane keeping

Lane keeping
1. Course Content
2. Running the Example Program
3. Source Code Analysis
3.1 auto_drive.py

1. Course Content

Note: This program runs on track map; personal maps require manual adjustments.

Start the autonomous driving program and use the lane-keeping and centering automatic driving functions on the track map.

2. Running the Example Program

Autonomous driving source code path: `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py`

- Start the camera node

```
ros2 launch ascamera hp60c.launch.py
```

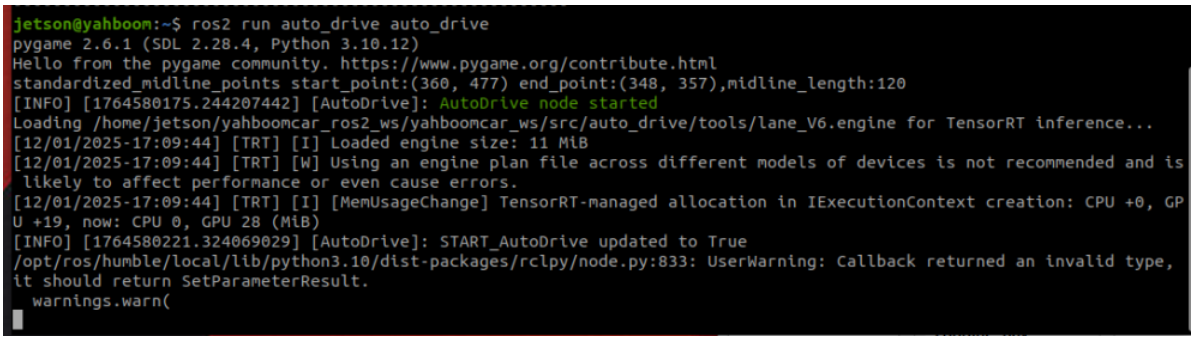
- Start the car chassis

```
ros2 run yahboomcar_bringup Mcnamu_driver_M1
```

- Start the lane centering program

```
ros2 run auto_drive auto_drive
```

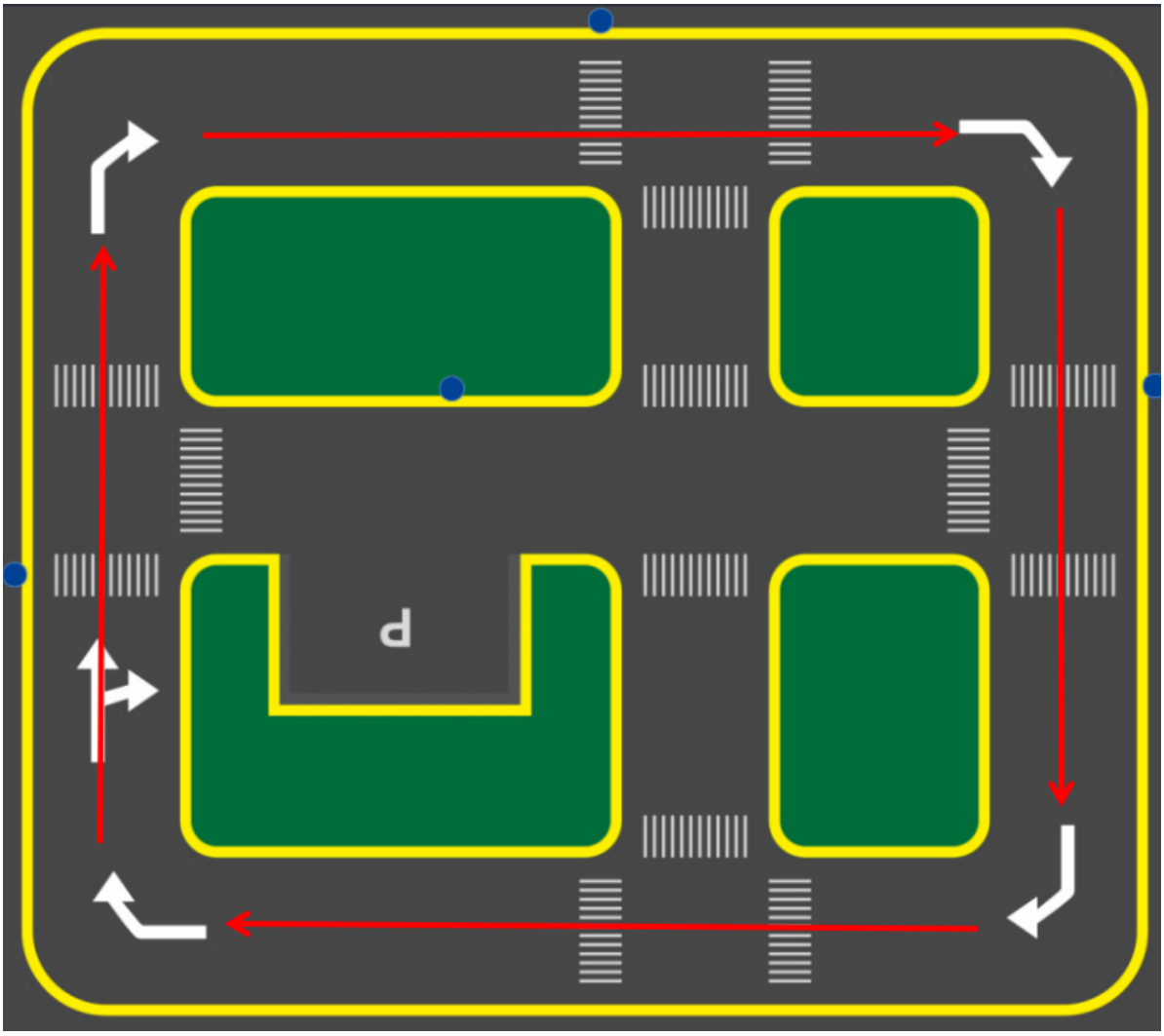
A visual interface will appear upon successful execution.



After successful startup, the car will normally drive directly along the center line of the sand table lane. If the car doesn't move after startup, you can adjust the dynamic parameters below to see if the car's operation parameters are turned on or off (START_AutoDrive = True, conbine_nav = False).

- The car will then automatically drive along the outer lane.

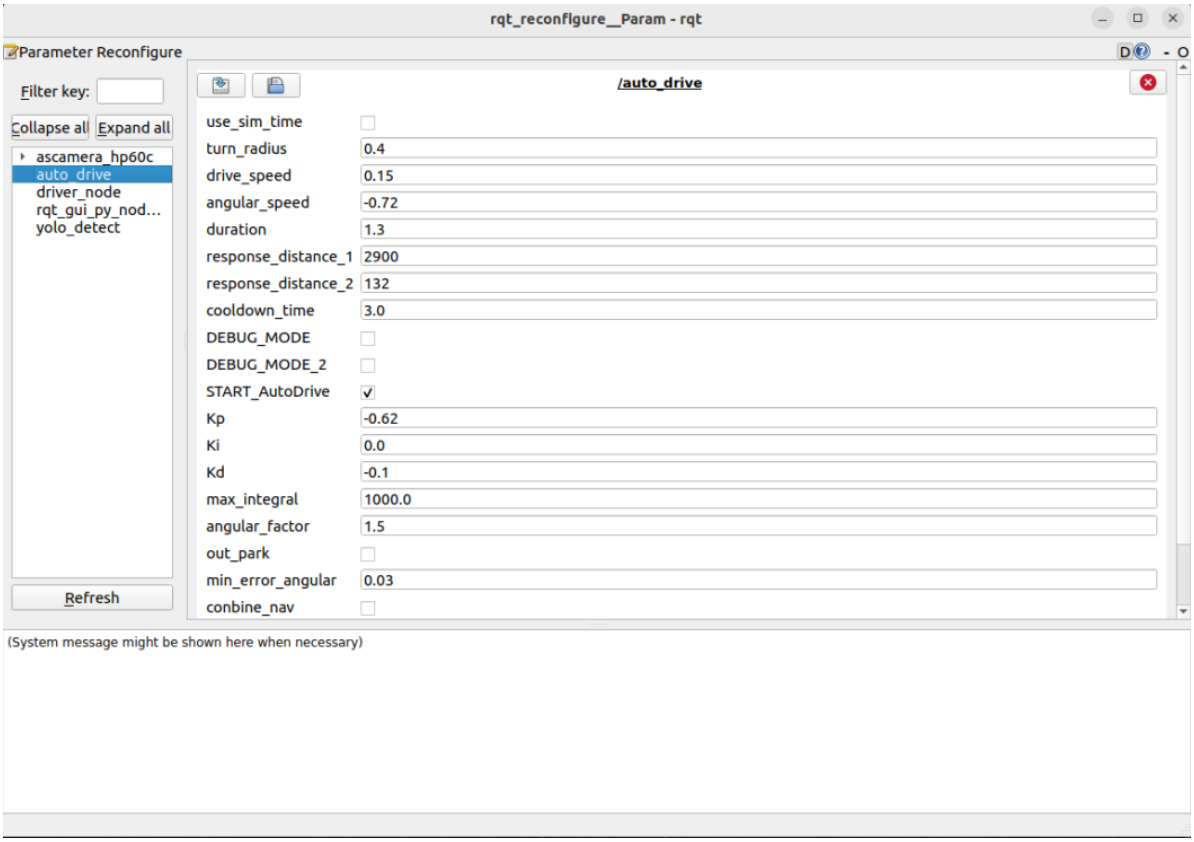
- On straight sections, it will automatically perform straight driving and lane centering.
- On curved sections, it will automatically steer and resume lane keeping mode after exiting the curve.



- To stop, simply uncheck `START_AutoDrive`.
- Start the dynamic parameter configuration node

```
ros2 run rqt_reconfigure rqt_reconfigure
```

You can optimize the car's lane centering effect by adjusting the above parameters in real time.



☑ After modifying the parameters, click the blank space in the GUI to enter the parameter values. Note that this will only take effect for the current startup; for permanent effects, the parameters need to be modified in the source code.

🛠 Parameter Explanation:

【turn_radius】: Turning radius. Normally, this parameter does not need to be changed.

【drive_speed】: Linear velocity. The car's driving speed. The factory default is 0.15. If you modify this parameter, you need to manually adjust other parameters.

【angular_speed】: Turning angular velocity.

【duration】: Right turn time for recognizing right turn signs.

【cooldown_time】: Road sign recognition cooldown time.

【DEBUG_MODE】: Visual debug parameter. Enabling this may affect centering.

【START_AutoDrive】: Controls the car's movement parameters. True means the car can move.

【kp, ki, kd】: Car centering PID parameters.

【conbine_nav】: Required only for road network navigation + YOLO enabled. Enabling this will stop the car.

The parameters above are the main adjustment and optimization parameters; other parameters should be kept at factory default values.

- Centerline Adjustment

If the car turns too early or too late during lane keeping, you can adjust the centerline to fine-tune the turning timing.

Parameter path:
`/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/midline_points.yaml`

```
midline_points:
  start_point:
    x: 360
    y: 477
  end_point:
    x: 355
    y: 424
  midline_length: 120
  mid_distance: [200,40]
```

The turning timing of the car can be adjusted by modifying the `midline_length` parameter. Taking a length of 120 as an example, if the turn is too early, we can try changing 120 to 115 to fine-tune it. Alternatively, we can adjust the turning timing by slightly lowering the camera angle; this will also delay the turn. The specific adjustments should be made based on the actual environment.

After making the modifications, the changes need to be compiled before running the program.

```
cd yahboomcar_ros2_ws/yahboomcar_ws/
colcon build --packages-select auto_drive
```

3. Source Code Analysis

3.1 auto_drive.py

The program source code is located at `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py`

Control Flow is divided into:

1. Receiving and processing image data
2. Detecting lane lines and traffic signs
3. Determining the control strategy based on the detection results
4. Issuing control commands (speed, direction)
5. Repeating the above steps.

```
def __handle_lane_detect_result(self,left_box,right_box,frame):
        '''Processing lane detection results'''
        self.error_mode = 99
        error_angular = [0.0, 0.0]  # 默认角度偏差Default angle deviation

        if left_box is not None and right_box is not None and left_box[0]
<right_box[0] :
            #If both left and right lane lines exist, draw the left and right
lane lines and their intersection.
            self.error_mode=0  #Marking error modes: Centering mode: 0, Left-
skewed mode: 1, Right-skewed mode: 2
            frame=self.lane_detect.draw_left_line(frame, left_box)#Draw left
lane line
            frame=self.lane_detect.draw_right_line(frame, right_box)#Draw the
right lane line

 frame,x_center,y_center=self.lane_detect.draw_center_point(frame,left_box,right
_box)# Draw lane estimation center point
```

```
 left_lane_offset,right_lane_offset=self.lane_detect.cal_lane_lateral_offset(lef
t_box,right_box)
            # self.get_logger().info(f"left_lane_offset: {left_lane_offset:.2f}
, right_lane_offset: {right_lane_offset:.2f} ")

 error_angular=self.lane_detect.cal_error_angular(x_center,y_center)#Calculate
the angular error between the vehicle body and the center point

        elif left_box is not None and right_box is None and left_box[0]<320:
            self.error_mode=1
            self.lane_detect.prev_center=None
            frame=self.lane_detect.draw_left_line(frame, left_box)
        elif right_box is not None and left_box is None :
            self.error_mode=2
            self.lane_detect.prev_center=None
            frame=self.lane_detect.draw_right_line(frame, right_box)
        else:
            self.lane_detect.prev_center=None

        # When the steering action is activated, lane speed control is paused.
        if self.turn_action_active:
            return frame

        if self.error_mode==0 :
            #Center
            if error_angular[1]>=0.03 or error_angular[1]<=-0.03:
                if abs(error_angular[1])<0.3:
                    angular_z=self.pid_controller.pid_control(error_angular[1])
                    angular_z=min(angular_z,0.3)
                    angular_z=max(angular_z,-0.3)

 self.__set_current_speed(linear_x=self.drive_speed,angular_z=angular_z)


        elif self.error_mode==1:
            #Only the left lane line
            if_infer=self.lane_detect.IF_lane_intersection(left_box)
            if if_infer is not None:

 self.__set_current_speed(linear_x=self.drive_speed,angular_z=self.angular_speed
)
            else:

 self.__set_current_speed(linear_x=self.drive_speed,angular_z=0.0)
        else:
            self.__set_current_speed(linear_x=self.drive_speed,angular_z=0.0)

        # Display angular deviation on the image
        cv2.putText(frame, f"Angle Dev: {error_angular[1]:.2f} rad", (10, 90),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 255), 2)
        frame=self.lane_detect.draw_midline(frame)
        return frame
```

The above is the key lane line centering procedure, which judges the actual turning time and the timing of centering based on the slope angle between the left and right yellow lane lines and the center line.